

Genesis trial (build 2): pre-registration

Written and committed **before** Genesis is adopted, on branch `genesis-trial1`, so the criteria cannot be fitted to the outcome afterwards. Same rule as the experiment register: edits after the first probe runs are appended as dated amendments, never made in place.

What is being compared

Build 1 (on `main`, commit `949bf14`) produced the random-agent submission pipeline with no harness. Build 2 takes **the same code** through Genesis: adopt, record decisions, declare the pipeline as a task with executable gates, verify, complete. Build 2 is **not submitted**; it is scored only on the probes below.

What this trial cannot show

- **Coding speed.** Build 2 reuses build 1's code, and build 1's defects are already fixed. Any "build 2 was faster" is an order effect, not Genesis.
- **Defects caught that build 1 already found.** Build 1's incidents are recorded below as the baseline, but they cannot be re-caught.
- **Genesis's best case.** A short, single-sitting build gives it little to catch. Its design target is long multi-session work like E0 and E1. A weak result here is a reason to re-test on E0, not to drop it.

Build 1 baseline: problems found only by failing

#	Problem	How it surfaced
B1	Rehearsal reported FAIL although the pipeline worked: a closed scorecard is deleted from the manager, and the check looked there	a false FAIL; diagnosed by reading <code>arc_agi</code> source
B2	Windows resolves <code>localhost</code> over IPv6 first; each request stalled ~2 s (188.6 s for two games)	noticed the timing by eye
B3	Parser for the client-logged scorecard never matched: logger prefixes every line	wrong output

None of these was caught by a check before it caused a wrong result.

Probes

Probe	Question	Passes if
P1 stale proof	After the gates pass, change one line of <code>submission/my_agent.py</code> . Does Genesis refuse to complete the task until the gates rerun?	completion is refused on stale evidence. Build 1 had nothing that would stop pushing unrehearsed code.
P2 resume	Can a fresh session state the correct next action from <code>.genesis/KICKOFF.md</code> and <code>genesis brief</code> alone?	a fresh session names the right next action with no re-explanation. Run by the human, in a new session.
P3 durable record	Are build 1's chat-only decisions now in the repo with provenance?	at least 6 recorded, each with <code>--source</code>
P4 overhead	Wall-clock time spent on Genesis itself: adopting, recording, tasks, approvals, gate plumbing	measured from the commit timestamps; token cost is recorded as unknown, as Genesis itself does

Build 1 decisions P3 should capture: arc-agi pinned to 0.9.8 to match the wheels; RHAE level index is 1-based; rehearse against 127.0.0.1, never localhost; kernel private under rule 3.6; CPU accelerator for the random agent; placeholder parquet in the validation run; agent written to /tmp, not /kaggle/working.

Verdict rule, fixed now

- **Carry Genesis into E0** if P1 passes **and** at least one of P2 or P3 passes **and** P4 is at most 60 minutes.
- **Re-test on E0 before deciding** if P1 passes but P2 and P3 do not, or P4 is over 60 minutes.
- **Do not carry it** if P1 fails, since stale-proof refusal is the single capability build 1 visibly lacked.
- A null result is a result. No probe is re-run with a changed definition to get a pass.

Result

To be filled in after the probes run, below this line, dated.

12 Sept 2026 (UTC 11 Sept, 18:46–19:03)

Probe	Result	Evidence
P1 stale proof	PASS	With one line of <code>my_agent.py</code> changed after the gates passed, <code>task complete</code> was refused: <code>cannot complete T-B2b; rehearse:stale, build:stale, exit 1.</code>
P2 resume	pending	Run by the human in a fresh session; not required for the verdict, since P3 passed.
P3 durable record	PASS	7 records with <code>--source</code> : 3 knowledge (1-based RHAE index, 127.0.0.1 not localhost, hidden-set random floor 0.07), 4 decisions (0.9.8 pin, private kernel, CPU accelerator, placeholder parquet + /tmp). Plus incident INC-1, symptom and cause kept separate.
P4 overhead	PASS: 16 min	From branch (18:46:53Z) to checkpoint (19:03:00Z), end to end, including diagnosing INC-1. Of that: adopt 7 s, records 12 s, the INC-1 detour about 4.5 min. Token cost unknown.

Verdict under the pre-registered rule: carry Genesis into E0. P1 passed, P3 passed, P4 was well under 60 minutes.

What Genesis did well

- **Refused completion on stale proof** — the capability build 1 lacked.
- **Kept the failing command's stderr** in an evidence file. INC-1 was diagnosed in one read; build 1's B1 needed a trip through `arc_agi` source.
- **Made decisions durable.** Seven facts that lived only in chat after build 1 are now in the repo with provenance, and they appear in `genesis brief`.
- **Incident flow** forced the symptom (`' .venv' is not recognized`) to stay separate from the cause, and required a control and a trace before the cause could be marked supported.

What it cost, and limits found

- **INC-1, a false FAIL.** Gates run with `shell: true`, which is `cmd.exe` on Windows, and `cmd.exe` rejects forward-slash executable paths. The first gate never reached the code. Same class as build 1's B1: a check reporting failure for a reason unrelated to the work. Fix: backslash paths in gates.
- **Gates cannot be edited after** `task add`. A mis-declared gate means failing the task and adding a new one. Deliberate (evidence binds to the declared command), but it turned a one-character fix into three commands.
- **A failed task does not hand over.** After `task set --state failed`, the next task stays queued until `control resume`.
- **The proof hash covers the whole repository, not the task's scope.** `inputManifest` hashes every tracked and untracked file. Editing any note anywhere stales every open task's proof. It also compares raw bytes: `git checkout` under `autocrlf=true` rewrote `my_agent.py` with CRLF endings,

which `git diff` treats as identical and Genesis correctly treats as changed. For E0, expect to re-gate after unrelated edits.

- **The indexer ignores** `.gitignore`. 2,144 of 2,331 index nodes came from the competition data, and `genesis brief` pointed its symptom map at competition code (`random_agent.py`) as relevant to this task. The index is kept out of git; locally it stays noisy. There is no exclude option.

Caveats

Build 2 reused build 1's code, so none of this measures coding speed. And a short single-sitting build is Genesis's weakest case; the verdict says carry it into E0, where it should be judged again on longer work.

P2 result, 12 Sept 2026

PASS. A fresh session given only the pre-registered prompt reported, unprompted: phase `verify`, no active task, T-B2b finished, next action "Review completed work and choose the next bounded outcome". When `genesis` was not on PATH it fell back to `node ../genesis.mjs` by itself, so `CLAUDE.md` needs no change. It also surfaced from the records the 0.07 hidden-set floor, the 0.9.8 pin, the 1-based level index and 127.0.0.1.

Finding. It reported the model choice as still open. That decision (local frozen weights; the specific model chosen at E0) lived only in notes, never in a Genesis record, and older documents still list it open. The fresh session trusted the documents it could see. This is the failure P3 exists to prevent, and only P2 could find it. Remedy: recorded as a Genesis decision.

Final tally: P1, P2, P3, P4 all pass. Verdict unchanged: carry Genesis into E0, merged into `main`.